

LLM-ERP Operation Manual

LLM-Powered Enterprise Resource Planning System
Version: v0.1.0 | Updated: 2026-05-07

1. System Overview

LLM-ERP is an open-source intelligent ERP system that lets you manage your entire factory floor through **natural language** — no menu clicking, no T-codes to memorize. It covers **12+ modules, 25 business constraints**, and **37 LLM-powered tools**:

Module	Code	Core Function
 Bilingual NL	—	Chinese OR English. System auto-detects your language.
 Inventory	MM	Part management, stock query, inbound/outbound transactions, location tracking
 Purchasing	PP	Supplier management, PO lifecycle, supplier scoring
 BOM Engineering	ENG	Product structure, multi-level explosion, shortage checking
 Dispatch	MFG	Work order management, machine scheduling, dynamic rescheduling (3 strategies)
 Quality	QM	Inspection orders, non-conformance tracking, CAPA
 Accounting	FI	Chart of accounts, journal entries, AR aging, month-end close
 Reports	—	Generate PDF reports via natural language (inventory, AR, purchase, production, P&L)
 CRM	SD	Customer master with A/B/C grading, sales orders (SO), interaction events, sales role dashboard
 War Room	—	SVG value-stream dashboard, real-time event animations, multi-screen display
 7 Roles	—	Plant Manager / Production Planner / Warehouse / Purchasing / QA / Accounting /  Sales
 Leads & Opportunities	CRM	Lead source tracking, scoring, pipeline stage management, conversion analytics
 Contract Management	SD	Framework contracts, annual agreements, project contracts, auto-pricing on SO, expiry alerts
 Decision Log & AAR	QM	Auto-log major decisions, After Action Review workflow, KPI feedback loop
 Cash Flow & Rush Orders	FI	Financial impact evaluation, 30-day cash projection, auto-block PO on insufficient cash
 3 Factory Types	MFG	MTO / MTS / ETO — auto-adjusts pipeline, form fields, and cash flow rules

2. Requirements

Hardware Requirements

Setup	Minimum	Recommended
RAM	4 GB	16 GB (with local LLM)
CPU	2 cores	8 cores
Local LLM	—	16 GB RAM + GPU (optional)

Software Requirements

- Python 3.11+
- Node.js 18+
- (Optional) Docker Desktop — for PostgreSQL production environment
- (Optional) Ollama — for local LLM inference

3. Installation

3.1 Download the Project

```
git clone https://github.com/fanchanyu/llm-erp.git
cd llm-erp
```

3.2 Backend Configuration

```
cd backend
cp .env.example .env
# Edit .env with your LLM Provider's API Key
```

`.env` File Reference:

Variable	Description	Example
LLM_PROVIDER	LLM provider	deepseek / ollama / anthropic / openai
LLM_MODEL	Model name	deepseek-chat / gemma4:e4b / claude-sonnet-4
MAX_TOOL_ROUNDS	Max tool call rounds	5 (cloud) / 8-10 (local)
DEEPSEEK_API_KEY	DeepSeek API Key	sk-xxxx

3.3 Getting an API Key (Choose One Provider)

LLM-ERP needs an LLM (Large Language Model) to power its natural language interface. Below are sign-up guides for each provider:

♦ **DeepSeek (Recommended — cheapest)**

1. Go to https://platform.deepseek.com/sign_up and register
2. Verify email → Log **in**
3. Click "API Keys" on the left sidebar → "Create API Key"
4. Copy the key (format: sk-xxxxxxxxxxxxxxxxxx)
5. Set **in** `.env`: `DEEPSEEK_API_KEY=sk-xxxx`

- **Cost:** ~~¥1 CNY processes 200~~ 500 queries
- **Model:** deepseek-chat
- **Best for:** Getting started, development phase

♦ Anthropic Claude (Smartest)

```
1. Go to https://console.anthropic.com/ and register
2. Log in → Click "API Keys" → "Create Key"
3. Copy the key (format: sk-ant-xxxxxxxxxxxx)
4. Set in .env:
  LLM_PROVIDER=anthropic
  LLM_MODEL=claude-sonnet-4
  ANTHROPIC_API_KEY=sk-ant-xxxx
```

- **Cost:** \$3/M input + \$15/M output tokens
- **Models:** claude-sonnet-4 (recommended) / claude-haiku-3-5 (fast & cheap)
- **Best for:** Production environment, high accuracy needs

♦ OpenAI GPT (Most popular)

```
1. Go to https://platform.openai.com/signup and register
2. Log in → Click top-left → "API Keys" → "Create new secret key"
3. Copy the key (format: sk-proj-xxxxxxxxxxxx)
4. Set in .env:
  LLM_PROVIDER=openai
  LLM_MODEL=gpt-4o
  OPENAI_API_KEY=sk-proj-xxxx
```

- **Cost:** \$2.50/M input + \$10/M output (gpt-4o)
- **Models:** gpt-4o (recommended) / gpt-4o-mini (budget)
- **Best for:** Existing OpenAI users, general purpose

♦ OpenRouter (One endpoint for many models)

Provides a unified API endpoint for multiple models including DeepSeek, Claude, GPT, and more:

```
1. Go to https://openrouter.ai/keys and register
2. Log in → "Create Key"
3. Copy the key (format: sk-or-v1-xxxxxxxxxx)
4. Set in .env:
  LLM_PROVIDER=openrouter
  LLM_MODEL=deepseek/deepseek-chat
  OPENROUTER_API_KEY=sk-or-v1-xxxx
```

- **Cost:** Varies by model (you can pick the cheapest)
- **Model list:** <https://openrouter.ai/models>
- **Best for:** Comparing multiple models, easy switching

♦ Ollama (Local model, completely free, no API Key needed)

If your computer has 16GB+ RAM, you can run models locally with zero API costs:

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Download Gemma4 model (8B, ~9.6GB)
ollama pull gemma4:e4b
```

```
# Verify Ollama is running
curl http://localhost:11434/api/tags

# Set .env:
# LLM_PROVIDER=ollama
# LLM_MODEL=gemma4:e4b
# MAX_TOOL_ROUNDS=8
```

- **Cost:** Completely free — just need hardware
- **Hardware:** 16GB RAM (8B model) / 32GB (12B model)
- **Best for:** Data privacy, no API costs

⚠ **API Key Security: Keys must only go in `.env`. NEVER put them in CSV data files, Git commits, or post them on GitHub.**

3.8 Start the Backend

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload
```

The backend starts at `http://localhost:8000`.

3.9 Start the Frontend

```
cd frontend
npm install
npm run dev
```

The frontend starts at `http://localhost:5173`.

3.10 Start Local LLM (Optional)

If you chose Ollama, first follow [Section 3.3 — Ollama](#) to install Ollama and download the model, then verify:

```
curl http://localhost:11434/api/tags
```

Then go back to [Section 3.8](#) to start the backend.

3.11 Data Management (Import / Export / Reset)

LLM-ERP provides a standardized data import/export tool. All data uses **natural keys** (part numbers, supplier names, product codes) — not UUIDs — so CSV files are human-readable and directly editable. **API keys are NEVER stored in data files**, only in `.env`.

Import Data

```
cd backend

# Import built-in sample data (12 tables, 71 records)
python -m scripts.manage_data import scripts/sample_data/
```

```
# Or use your own CSV
python -m scripts.manage_data import path/to/my-data.csv

# Or import an entire directory (auto-sorted by dependency order)
python -m scripts.manage_data import path/to/data-dir/

# Dry run (validate without modifying DB)
python -m scripts.manage_data import scripts/sample_data/ --dry-run
```

Export Data

```
python -m scripts.manage_data export ./backup/
# Outputs: 01-parts.csv ~ 12-ar.csv, 12 tables total
```

Reset Database

```
python -m scripts.manage_data reset --force
# Clears all 22 tables
```

View Schema

```
python -m scripts.manage_data schema
# Lists all entity field definitions
```

CSV Schema Quick Reference

File	Entity	Required Fields	Depends On
01-parts.csv	Parts Master	part_no, name, unit	—
02-suppliers.csv	Suppliers	name	—
03-products.csv	Products	product_no, name	—
04-work-centers.csv	Work Centers	name	—
05-accounts.csv	Chart of Accounts	account_no, name, type, normal_balance	—
06-inventory.csv	Stock Levels	part_no, location, quantity	parts
07-bom.csv	Bill of Materials	product_no, part_no, quantity, level	products, parts
08-purchase-orders.csv	Purchase Orders	po_no, supplier_name, item_part_no, item_quantity	suppliers, parts
09-production-orders.csv	Production Orders	order_no, product_no, quantity, due_date	products
10-quality.csv	Inspection Orders	inspection_no, part_no, quantity	parts
11-accounting.csv	Journal Entries	entry_no, description, entry_date, period, line_account_no, line_debit, line_credit	accounts

12-ar.csv	Accounts Receivable	customer_name, invoice_no, amount, due_date	—
-----------	---------------------	---	---

The import tool handles dependency ordering automatically and skips existing records (idempotent).

4. Usage Guide

4.1 Inventory

Ask anything about stock:

English Query	Description
"How many M6x20 screws are in stock?"	Query specific part stock
"Show me the full inventory list"	List all stock
"Which transmission parts are low on stock?"	Category-based query
"Check motor parts stock status"	Fuzzy name search
"Receive 500 M6x20 screws"	Inbound transaction
"Issue 100 M6x20 screws to work order WO-001"	Outbound/picking

4.2 Purchasing

English Query	Description
"List all suppliers, check DaMing Screws"	Query suppliers
"Create a PO to DaMing Screws for 200 M6x20 screws"	Create purchase order
"List all current purchase orders"	Query PO list
"Has PO-2026-0001 arrived?"	Check PO status

4.3 BOM Management

English Query	Description
"What parts does CNC-001 use?"	Query BOM structure
"Show me the CNC-001 BOM explosion"	Multi-level BOM explosion
"Can I build 5 CNC-001 units? Check material availability"	Shortage check (core feature)

4.4 Production Dispatch

English Query	Description
"List all current work orders"	Query work orders
"Release work order WO-20260506-001"	Release work order
"CNC-01 is down, right-shift 30 minutes"	Right-shift reschedule
"CNC-01 is down, switch to backup machine"	Route change reschedule
"Emergency order WO-002, expedite it"	Expedite (insertion)

4.5 Quality Management

English Query	Description
"Create inspection for M6x20, lot LOT-001"	Create inspection order
"Show inspection records"	Query inspection list
"Are there any NCs for M6x20?"	Query non-conformances
"Create NC for M6x20, dimension out of spec, severity=Major"	Create NC record

4.6 Accounting

English Query	Description
"Which AR invoices are overdue?"	Query overdue AR
"Show me the AR aging report"	Query AR list
"List chart of accounts"	Query accounts
"Create journal entry: debit Inventory 1000 credit Bank 1000"	Create journal entry

4.7 CRM (Sales)

English Query	Description
"Show all customers, search YongYu"	List/search customers
"Add customer: HongDa Electronics, contact Mr. Zhang"	Create new customer
"Create SO for YongYu: CNC-001 x 10 units"	Create sales order
"List current sales orders"	Query SO list
"Confirm SO-20260507-001"	Confirm SO → auto-create production order
"Ship SO-20260507-001"	Ship SO → auto-deduct inventory
"Log: YongYu called about delivery date"	Create customer interaction event
"Show conversation history for YongYu"	Query customer chat history

4.8 Cross-Module Queries

English Query	Description
"Has PO-0001 for M6x20 screws arrived? Has it been received?"	Purchasing + Inventory
"I need to build 5 CNC-001 units — first check if materials are available"	BOM + Purchasing

4.9 Lead & Opportunity Management

Track potential customers from first contact through deal closure.

Feature	Description
Lead Sources	Exhibition / Website / Referral / Cold Call — auto-tagged on creation

Lead Scoring	Auto-calculated score based on source, budget, timeline, and interaction frequency
Pipeline Stages	New → Qualified → Proposal → Negotiation → Won/Lost
Conversion Analytics	Win rate, average deal cycle, source effectiveness breakdown

Sample LLM Queries:

English Query	Description
"Show me my leads"	List all leads with status and score
"What's in the pipeline?"	Show opportunities by pipeline stage
"Create a lead from the Beijing exhibition"	Add a new lead with source tag
"Show conversion rate by source"	Analytics: which sources close best

4.10 Contract Management

Manage customer and supplier contracts with auto-enforcement on transactions.

Contract Type	Description
Framework Contract	Long-term pricing agreement with preferred customer
Annual Agreement	Yearly supply contract with volume commitments
Project Contract	One-time project with fixed scope, milestones, and payment terms

Key Features:

- **Auto-apply contract pricing:** When creating a SO for a contracted customer, the system automatically applies the contract unit price without manual lookup
- **Expiry alerts:** System notifies 30 days before contract expiration
- **Status tracking:** Active / Expiring / Expired / Renewed

Sample LLM Queries:

English Query	Description
"List active contracts"	Show all currently active contracts
"When does YongYu's contract expire?"	Query specific contract expiry date
"Create a new annual contract for HongDa Electronics"	Add a contract with terms
"Show contracts expiring this month"	Proactive expiry management

4.11 Decision Log & After Action Review (AAR)

LLM-ERP automatically logs major decisions and provides a structured post-mortem workflow.

Auto-Logged Decisions:

Decision Type	Trigger
Rush order acceptance	Rush order assessment executed
Supplier change	Supplier switch on a PO or work order

Schedule change	Reschedule operation (right-shift, route change)
Price override	Manual price adjustment on SO or PO

AAR Workflow:

① Expected vs Actual → ② Variance Analysis → ③ Corrective Action → ④ Rule Update

- **Expected vs Actual:** Compare planned outcome with actual result (cost, timeline, quality)
- **Variance Analysis:** Identify root causes of deviation
- **Corrective Action:** Generate actionable improvement items
- **Rule Updates:** Optionally convert learnings into new system constraints

Department KPI Feedback Loop: Corrective actions from AAR feed into department KPIs (e.g., if rush orders cause quality issues, QA KPI is adjusted).

Sample LLM Queries:

English Query	Description
"Show recent sales decisions"	List recent auto-logged decisions in sales
"AAR for last month's rush orders"	Run AAR on rush order decisions from last month
"What corrective actions are still open?"	Check pending AAR action items
"Show KPI changes after the AAR"	View KPI impact from completed reviews

4.12 Rush Order Assessment & Cash Flow Constraints

Evaluate financial impact of rush orders in real time and enforce cash-based controls.

Financial Impact Evaluation Formula:

Net Impact = Premium Revenue + Overtime Cost + Delay Penalties

- **Premium Revenue:** Additional charge for rush service (e.g., +20% on unit price)
- **Overtime Cost:** Additional labor and machine hours
- **Delay Penalties:** Penalty from pushing existing orders (if applicable)

Cash Flow Features:

- **Cash position query:** Display current cash balance, AR, AP, and net position
- **30-day cash projection:** Forecast inflows (AR collections) vs outflows (AP due, payroll)
- **Auto-block PO creation:** If cash position is insufficient to cover the rush order cost, PO creation is blocked with an explanation

Sample LLM Queries:

English Query	Description
"Evaluate this rush order"	Run financial impact assessment
"What's our cash position?"	Show current cash + 30-day projection
"Create a rush order for YongYu: 5 CNC-001 in 3 days"	Attempt rush SO creation (may be blocked)

4.13 Factory Type Configuration

LLM-ERP supports three factory types that automatically adjust pipelines, form fields, and business rules.

Type	Description	Typical Workflow
MTO	Make-to-Order	Lead → Opportunity → Quote → SO → BOM → Work Order → Produce → Ship
MTS	Make-to-Stock	Forecast → Production Plan → Produce → Stock → SO → Ship
ETO	Engineer-to-Order	RFQ → Design → Quote → Contract → Milestones → BOM → Produce → Bill

What Auto-Adjusts:

- **Pipeline stages:** Different stages for each factory type
- **Form fields:** Additional fields appear (e.g., design specs for ETO)
- **Cash flow rules:** MTS may have inventory holding costs, ETO has milestone-based billing
- **AAR triggers:** Different decision types are auto-logged per type

Configure via API or Natural Language:

```
# Set factory type via LLM
You → "Set factory type to ETO"
System → "Factory configured as ETO. Pipeline stages updated to: RFQ → Design → Quote → Contract
→ Milestones → Produce → Ship"
```

```
# Or via curl (direct API)
curl -X PUT http://localhost:8000/api/factory/config \
  -H "Content-Type: application/json" \
  -d '{"factory_type": "mto}"'
```

5. War Room

The War Room is a full-screen SVG value-stream dashboard, suitable for factory floor multi-monitor setups.

How to open:

```
# After starting the frontend, open in browser:
http://localhost:5173/war-room.html
```

War Room features:

- 🏠 Horizontal 6-stage layout: Supplier → Purchasing → Inventory → Dispatch → Quality → Accounting
- 🟦 Animated particle flow: Green = material flow, Yellow = cash flow
- 📡 Bottom event stream: Shows the latest 30 events in real time
- 🔔 Node glow: Event-triggered visual feedback on corresponding stage
- ▶ Simulate events button: Demo-mode auto event generation
- 🔄 Auto-refresh every 15 seconds

6. Provider Switching

LLM-ERP supports 5 LLM Providers — switch anytime:

```
# Edit backend/.env
LLM_PROVIDER=deepseek           # or ollama / anthropic / openai / openrouter
LLM_MODEL=deepseek-chat         # or gemma4:e4b / claude-sonnet-4 / gpt-4o
MAX_TOOL_ROUNDS=5               # cloud=5, local=8-10
```

Switching principles:

- **Cloud API** (DeepSeek / Claude / GPT): MAX_TOOL_ROUNDS=5, needs API Key
- **Local model** (Ollama / Gemma4): MAX_TOOL_ROUNDS=8-10, no API Key needed
- **Bilingual**: All providers support Chinese AND English. Use the language you prefer.
- Backend auto-reloads on `.env` change (`uvicorn --reload`)

Provider Benchmark (30-test)

Metric	DeepSeek (Cloud)	Gemma4 8B (Local CPU)
	Pass Rate	90% (27/30)
	Avg Response	7.7s
	Cost per Test	~\$0.002
	Data Sovereignty	External API

7. Running the Benchmark

```
cd /mnt/d/Project/LLM_ERP/evaluation

# Run standard 30-test benchmark
python3 run_eval.py

# Verbose output
python3 run_eval.py --verbose

# Specify output file
python3 run_eval.py --output my-results.json
```

8. Frequently Asked Questions

Q: Backend won't start?

A: Make sure `.env` has a valid API Key. Make sure `pip install -r requirements.txt` has been run.

Q: LLM response is too slow?

A: DeepSeek averages 7.7s, which is normal. For local models (Gemma4), CPU inference takes ~8-10s; GPU can drop to 1-3s.

Q: Query results are wrong?

A: Try increasing `MAX_TOOL_ROUNDS` (local models need more rounds). You can also add specific constraints to the system prompt.

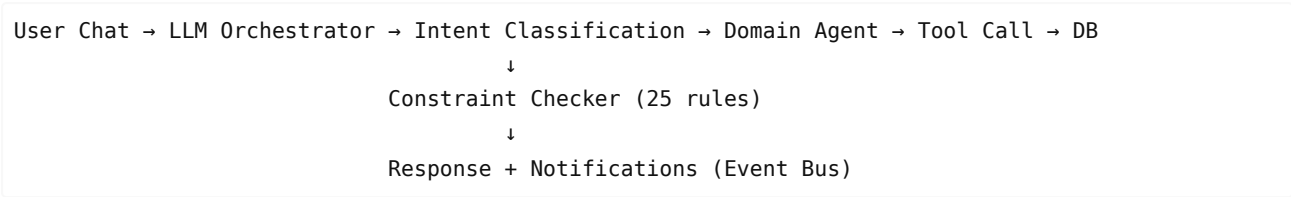
Q: How do I clear the database?

A: Use the built-in tool: `cd backend && python -m scripts.manage_data reset --force` . Or delete `backend/llm_erp.db` and re-import.

Q: Vite doesn't pick up new files in public/?

A: Restart Vite dev server: `Ctrl+C` then re-run `npm run dev` .

9. System Architecture Quick Reference



Layer	Technology	Description
Frontend	React 18 + TypeScript + Vite + Tailwind	Role dashboards, Chat UI, War Room
Backend	Python FastAPI + SQLAlchemy	7 domain services, 42 API routes
LLM	DeepSeek / Anthropic / OpenAI / Ollama	37 tool definitions, Function Calling
Events	Pub/Sub Event Bus	10 event types, role-based routing
Database	SQLite (dev) / PostgreSQL (prod)	22 tables, Alembic migrations

10. Scenario Walkthroughs

End-to-end workflow demonstrations showing how to use LLM-ERP in real situations.

Scenario 1: Order-to-Delivery (End-to-End)

Situation: Customer ordered 3 units of CNC-001, due by May 15.

Step	You Say	System Response
① Check stock	"Any CNC-001 in stock?"	Only 1 unit available
② BOM check	"I need to build 3 CNC-001, check material availability"	Shortage: 4 bearings, 200 screws needed
③ Create PO	"Order 4 bearings and 200 M6x20 screws from DaMing"	PO-20260507-001 created
④ Receive goods	"PO-20260507-001 arrived, receive 4 bearings, 200 screws"	Stock updated: bearings:8, screws:350
⑤ Dispatch	"Release work order for 3 CNC-001 units"	WO-20260507-001 released & scheduled
⑥ Complete	"WO-20260507-001 done, all 3 units finished"	3 CNC-001 received Work order closed
⑦ Check cost	"What's the actual cost?"	Materials \$X, Labor \$Y, Total \$Z

Scenario 2: Exception Handling — Machine Breakdown

Situation: CNC-01 machine broke down. Orders need rescheduling.

① Right-shift reschedule (fastest recovery)
You → "CNC-01 is down, right-shift all operations by 2 hours"
System → "All CNC-01 operations shifted +2h, estimated completion: 05/07 16:00"
② Route change (if backup machine available)
You → "CNC-01 down, switch all work to CNC-02"
System → "Switched to CNC-02, orders rescheduled"

```
# ③ Expedite (urgent order)
You → "Emergency order W0-20260507-001, expedite it"
System → "W0-20260507-001 moved to front of queue, ETA 3 days earlier"
```

Scenario 3: Cross-Module Decision Support

Situation: Evaluating whether to accept a large order.

```
# Comprehensive query
You → "Customer wants 5 CNC-001, due 05/20. Can we take it?"
System → "BOM check: materials available → 
          Capacity check: CNC-01 has 40hr free before 05/15 → 
          Purchase lead: bearings arrive 05/10 → 
          Result: Accept! Estimated completion: 05/16"
```

Scenario 4: Quality Issue Tracing

Situation: Customer reports quality issue with CNC-001.

```
# ① Check NC records
You → "Any quality issues recorded for CNC-001 recently?"
System → "Found 1 NC: Lot LOT-B001, bearing dimension out of spec"

# ② Create CAPA
You → "Create a corrective action for NC-20260505-001"
System → "CAPA created: Switch bearing supplier, add 100% incoming inspection"

# ③ Trace back to supplier
You → "Which supplier provided those bearings?"
System → "DaMing Screws, score dropped from 4.2 to 3.5"
```

Scenario 5: CRM — Sales Order to Delivery (Full Flow)

Situation: Sales manager got a new order from YongYu Electronics. Walk through from customer query to shipment.

```
# ① Query / Create customer
You → "Look up YongYu Precision's info"
System → "YongYu Precision Industrial, Grade-A customer, Contact: Manager Lin, Credit Limit: NT$500K"

# ② Create Sales Order
You → "Create SO for YongYu: CNC-001 × 5 units, unit price 1,500"
System → "SO-20260507-004 created (draft), total NT$7,500"

# ③ Confirm SO → auto-create work order
You → "Confirm SO-20260507-004"
System → "Confirmed → Auto-generated work order W0-20260507-003 (in production)"

# ④ Ship → auto-deduct inventory
You → "Ship SO-20260507-004"
System → "Shipped, CNC-001 inventory: 47 → 42 units"

# ⑤ Complete
You → "Complete SO-20260507-004"
```

System → "SO-20260507-004 delivered to customer. Order complete 

⑥ Log interaction (sales record)

You → "Log: YongYu's Manager Lin is very satisfied with delivery"

System → "📞 Customer interaction event recorded"

Scenario 6: MTO Machine Shop — Rush Order Handling

Situation: Customer needs urgent CNC parts. The factory runs MTO (Make-to-Order). Walk through lead tracking → rush assessment → contract → SO → production → shipment.

① Track lead → qualify opportunity

You → "Show me my leads"

System → "3 leads: Beijing exhibition (score 85), Website inquiry (72), Cold call (45)"

② Advance qualified lead to opportunity

You → "Move Beijing exhibition lead to opportunity, create quote for CNC-001 × 5"

System → "Quote Q-001 created, opportunity moved to Proposal stage"

③ Customer accepts → create contract

You → "Quote Q-001 accepted. Create project contract for Beijing Tech with milestone billing"

System → "Contract CT-001 created. Milestones: 30% deposit → 40% delivery → 30% acceptance"

④ Rush order assessment

You → "Customer needs 5 CNC-001 in 3 days. Evaluate as rush order"

System → "Rush assessment: Premium +20% (\$1,500), Overtime cost (\$800), Delay penalty (\$200)
Net impact: +\$500 → Recommended: accept with rush premium"

⑤ Create SO with contract pricing

You → "Create rush SO for Beijing Tech: CNC-001 × 5, apply contract CT-001 pricing"

System → "SO-20260508-001 created. Contract pricing applied: \$1,500/unit (list: \$1,200/unit)"

⑥ Confirm → auto-create work order with expedite

You → "Confirm SO-20260508-001 as rush"

System → "Rush SO confirmed. Work order WO-20260508-001 created and expedited to front of queue"

⑦ Ship

You → "Ship SO-20260508-001"

System → "Shipped. Inventory deducted. Rush order completed in 3 days 

Scenario 7: MTS Electronics — Contract Renewal & Replenishment

Situation: Annual contract with a key customer is about to expire. Factory runs MTS (Make-to-Stock). Walk through contract check → renewal → contract pricing → SO from catalog → shipment.

① Check contract expiry

You → "Show contracts expiring this month"

System → "1 contract expiring: HongDa Electronics Annual Agreement, expires 2026-05-31"

② Renew contract

You → "Renew HongDa Electronics annual contract for 2026-2027, same terms"

System → "Contract renewed. New expiry: 2027-05-31. Contract pricing active"

③ Customer places order – auto-apply contract price

You → "Create SO for HongDa: Power Supply PS-001 × 200 units"
System → "SO-20260508-002 created. Contract pricing auto-applied: \$85/unit (list: \$100/unit)"

④ Check stock for MTS fulfillment
You → "Do we have enough PS-001 in stock?"
System → "Stock: 150 units. Shortage: 50 units. Auto-triggering production order"

⑤ Ship from stock
You → "Ship SO-20260508-002"
System → "Shipped 200 units. Stock: 150 → 0 (50 units in production, ETA 3 days)"

Scenario 8: ETO Automation Equipment — Project Milestones

Situation: New custom automation equipment project. The factory runs ETO (Engineer-to-Order). Walk through RFQ → Design → Negotiation → Contract → Milestones → Billing → AAR.

① RFQ received
You → "Record RFQ from PrecisionTech for custom assembly line"
System → "RFQ-2026-001 created. Status: New. Moving to Design phase..."

② Design complete, send quote
You → "Design complete. Create quote for PrecisionTech: assembly line, \$250K"
System → "Quote Q-002 created. ETO pipeline: RFQ → Design → Quote → Negotiation"

③ Negotiate and close
You → "Customer counter-offer \$230K. Accept and create project contract with milestone billing"
System → "Contract CT-002 created.
Milestones:
└─ Milestone 1: Design approval → \$69K (30%)
└─ Milestone 2: Assembly complete → \$92K (40%)
└─ Milestone 3: Customer acceptance → \$69K (30%)"

④ Create BOM and work orders
You → "Create BOM for assembly line project, release work orders for Milestone 1"
System → "BOM created (47 parts). Work orders released for design phase. ETA: 2 weeks"

⑤ Milestone billing
You → "Milestone 1 complete. Bill \$69K to PrecisionTech"
System → "Invoice INV-001 created: \$69K. AR recorded. Due: Net 30"

⑥ Project complete → AAR
You → "Project complete. Run AAR comparing planned vs actual"
System → "AAR Results:
└─ Budget: \$230K planned vs \$245K actual (+\$15K, +6.5%)
└─ Timeline: 45 days planned vs 52 days actual (+7 days)
└─ Root Cause: Design revisions during assembly (+3 days)
└─ Corrective: Add design review gate before production release"

⑦ AAR feeds into department KPIs
You → "Apply AAR corrective actions to Engineering KPI"
System → "Engineering KPI updated: Add 'Design Review Completion Rate' metric. KPI adjustment logged."